

第2章 运算符与表达式

建议学时:3-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 Go 算术运算的确定性:有符号溢出是定义良好的回绕,除零是 panic(与 C 的 UB/崩溃对照)
- 理解 ++ / -- 在 Go 中是**语句**而非表达式, `arr[i++]` 是编译错误
- 掌握全部位运算符,特别是 C 没有的 `&^` (位清除)与 `<< / >>` 对无符号/有符号的不同语义
- 理解短路求值与"条件必须是布尔值"的强制约束
- 掌握 `==` 的比较规则:哪些类型可比较、NaN 的坑、结构体整体比较
- 掌握多重赋值、`:=` 作用域规则,以及**没有三目运算符**时如何用 if-else 表达
- 熟记运算符优先级表,理解 Go 规定"求值顺序从左到右"的确定性

💡 从 C 到 Go:本章主题如何迁移

C 的运算符体系庞大而危险:有符号溢出是未定义行为、`i++` 混在表达式里造成求值顺序 UB、隐式整型提升让类型悄悄变化、三目运算符 `?:` 嵌套难读。Go 对这套体系做了三件事:**删减、规范化、增加确定性**。

删减:没有三目运算符(官方立场:它常被滥用, if-else 更清晰); ++ / -- 只保留后缀形式且只能是独立语句;没有 ~ 一元取反(用 `^x` 表示)、没有 ! 之外的隐式真值转换(if (x) 在 C 里是 "x 非零",在 Go 里必须 `if x != 0`,而 x 必须是布尔)。

规范化:负数取模与 C 一致(截断向零, `-5 % 3 == -2`);有符号整数溢出不再是 UB,而是按补码确定性地回绕;整数除零在运行时 panic(可以 recover,第 9 章)。

增加确定性:表达式求值顺序严格从左到右(不用再背 C 的"序列点"规则);优先级表更小更直观;比较运算有类型系统的强约束, `slice == slice` 直接编译失败。

一句话主线: C 的运算符是"强大的瑞士军刀",Go 的运算符是"上锁的扳手"——能干所有正经活,但每个动作都在规则之内。

📦 前置知识自查

- C 的算术、位、逻辑、比较、赋值运算符全集及优先级
- 整数除法与取模、有符号溢出的 UB 概念、整型提升规则
- 短路求值: `&&` 与 `||` 的行为;位运算在标志位管理中的应用
- 三目运算符 `?:` 的使用习惯;复合赋值 `+=` 等

🚀 本章学习路线

1. **第一阶段:**§2.1-§2.3,把算术与位运算的"确定性规则"记牢,亲手运行溢出与取模示例
2. **第二阶段:**§2.4-§2.6,重点理解"语句 vs 表达式"的边界与无三目的替代写法
3. **第三阶段(实验):**§2.7-§2.8 与章末实验:表达式求值计算器,把整章运算符织进一个递归下降解析器
4. **验收标准:**能说出 5 处 "Go 运算符与 C 的差异",能默写优先级表的低到高顺序

本章知识导图

- §2.1 算术运算:溢出回绕、除零 panic、负数取模
- §2.2 自增自减:语句而非表达式
- §2.3 位运算:& | ^ &^ << >> 与标志位管理
- §2.4 逻辑运算与短路:条件必须是布尔
- §2.5 比较运算:可比较类型、NaN、结构体比较
- §2.6 赋值与多重赋值:没有三目运算符
- §2.7 优先级与求值顺序
- §2.8 特色运算符:<-、... 与通道方向

§2.1 算术运算:溢出回绕与除零 panic

C 中 `INT_MAX + 1` 是有符号溢出,属于 UB,编译器可能回绕、可能优化出你意想不到的行为。Go 的规定完全不同:

- **有符号与无符号溢出都是定义良好的**:按补码回绕。例如 `int8(127) + 1 == -128`。
- **整数除零在运行时 panic**;浮点除零不 panic,得 `+Inf / -Inf / NaN` (与 C 的浮点行为一致)。
- **负数取模与 C 相同**(截断向零): `-5 % 3 == -2`;注意这与 Python 的 `1` 不同。
- 整数除法结果仍是整数: `7 / 2 == 3`,小数部分直接丢弃(与 C 相同)。

示例 2-1 溢出的确定性行为

```
package main

import "fmt"

func main() {
    var a int8 = 127
    fmt.Println(a + 1) // -128:回绕,定义良好

    var u uint8 = 255
    fmt.Println(u + 1) // 0:无符号回绕

    fmt.Println(7 / 2)    // 3:整数除法
    fmt.Println(-5 % 3)   // -2:截断向零(与 C 一致)
    fmt.Println(7.0 / 2)  // 3.5:浮点除法

    // 整数除零:运行时 panic
    // fmt.Println(1 / 0)

    // 浮点除零:得到 Inf,不 panic
    fmt.Println(1.0 / 0.0) // +Inf
}
```

⚠ 易错点 2-1:整数除零不是编译错误,而是运行时 panic

常量表达式 `1/0` 编译期就报错,但 `x := 0; 1/x` 能通过编译,运行时才 panic。生产上除法的分母若来自外部输入,必须显式判断为零;依赖 panic 再 recover 是反模式。

★ 重点:溢出是回绕,不是错误

与 C 的 UB 相比,Go 的溢出"至少不会出现编译器魔法"。但回绕仍然是 bug: `time.Second * 30` 若换算成纳秒会溢出 32 位。生产规则:**金额、时间、ID 一律用 64 位并做边界检查**;计数器溢出问题用 `math.MaxInt64` 前检查或改用 `uint64`。

§2.2 自增自减:语句而非表达式

C 里 `i++` 是表达式,能出现在任何表达式位置: `arr[i++] = x`、`printf("%d", i++)`。Go 把 `++` / `--` 降级为**语句**:

- 只有后缀形式: `i++`、`i--`; `++i` 是语法错误
- 不能用在表达式里: `arr[i++]`、`f(i++)`、`a = i++` 全是编译错误
- 返回值没有意义,它只是"`i = i + 1`"的简写

示例 2-2 自增自减的正确与错误用法

```
package main

import "fmt"

func main() {
    i := 0
    i++
    i++          // 语句形式,合法
    fmt.Println(i) // 2

    // 以下全部编译失败
    // a := i++      // 不能赋值
    // arr := []int{}
    // fmt.Println(arr[i++]) // 不能嵌入表达式
    // ++i           // 没有前置形式
}
```

🚀 工程建议:循环里用 `i++` 语句,别怀念 C 的写法

一开始会不习惯 `for i := 0; i < n; i++ {}` 里的 `i++` 无法嵌入别的表达式。这是刻意的:它消灭了"后置副作用"这一整类求值顺序 bug。需要自增后取值的场景,拆成两行即可,可读性反而更好。

§2.3 位运算:& | ^ &^ << >>

Go 的位运算符与 C 大体相同,但有三个差异值得注意:

- `^` 一元是**按位取反**(相当于 C 的 `~`),二元是**异或**(与 C 相同);Go 没有 `~` 运算符
- `&^` (AND NOT)是 Go 独有:把左操作数中与右操作数都为 1 的位清零,等价于 `a & ^b`
- `>>` 右移:**无符号数逻辑右移,有符号数算术右移**(补符号位),与 C 中"有符号右移实现定义"相比更确定

示例 2-3 位运算全集

```

package main

import "fmt"

func main() {
    a := 0b1100 // 12
    b := 0b1010 // 10

    fmt.Printf("%04b & %04b = %04b\n", a, b, a&b) // 1000
    fmt.Printf("%04b | %04b = %04b\n", a, b, a|b) // 1110
    fmt.Printf("%04b ^ %04b = %04b\n", a, b, a^b) // 0110
    fmt.Printf("%^04b(按位取反,注意符号位) = %b\n", a, ^a)
    fmt.Printf("%04b &^ %04b = %04b\n", a, b, a&^b) // 0100:清除与 b 重合的位

    var x int8 = -8 // 二进制 11111000
    fmt.Println(x >> 1) // -4:算术右移,补符号位
    var y uint8 = 0b11111000
    fmt.Println(y >> 1) // 124:逻辑右移,补零
}

```

2.3.1 位运算生产场景:标志位管理

示例 2-4 用位掩码管理权限

```

package main

import "fmt"

// 权限位
const (
    Read  = 1 << iota // 1
    Write           // 2
    Exec           // 4
)

func has(p, flag int) bool { return p&flag != 0 }

func main() {
    perm := Read | Write // 授予读与写
    fmt.Println("可读?", has(perm, Read))
    fmt.Println("可写?", has(perm, Write))
    fmt.Println("可执行?", has(perm, Exec))

    perm = perm &^ Write // 撤销写权限(&^ 一键清除)
    fmt.Println("撤销后可写?", has(perm, Write))
    fmt.Println("权限值:", perm)
}

```

⚠ 易错点 2-2:用有符号数做位掩码,右移行为与直觉相反

标志位应使用无符号类型: `uint32` / `uint64`。若用 `int` 且最高位被置位, `>>` 是算术右移,会不断补 1,标志判断可能永远为真。 `&^` 清除标志时同样要保证类型一致。

§2.4 逻辑运算与短路:条件必须是布尔

`&&`、`||`、`!` 与 C 的短路语义完全一致:左侧能确定结果就不再求值右侧。但 Go 强制**操作数必须是布尔类型**——C 里 `if (p)`、`if (!buf)`、`while (n)` 的"隐式真值"在 Go 中一律编译错误,必须写 `if p != nil`、`if n != 0`。

示例 2-5 短路求值与显式布尔条件

```
package main

import "fmt"

func check(x int) bool {
    fmt.Println("check 被调用,参数:", x)
    return x > 0
}

func main() {
    a, b := 1, -1

    // 短路:左侧 false,右侧不执行
    if a > 10 && check(b) {
        fmt.Println("不会走到这里")
    }

    // 短路:左侧 true,右侧不执行
    if a > 0 || check(b) {
        fmt.Println("check 没被调用")
    }

    // 常见的非空判断链:利用短路保护
    var s []int
    if len(s) > 0 && s[0] == 42 {
        fmt.Println("首元素是 42")
    }

    // C 风格"非零即真"在 Go 中是编译错误
    // if a { ... }           // 错误:a 不是布尔
    // if s { ... }           // 错误:s 不是布尔
    _ = a
}
```

⚠ 易错点 2-3:忘记 Go 没有隐式真值转换

从 C 迁移时最频繁的编译错误: `if (p)` 请写 `if p != nil`; `if (n)` 请写 `if n != 0`。这不是麻烦,而是把"条件到底是什么"写进了代码,配合 `go vet` 可以提前抓住"函数返回了 int 却当布尔用"的 API 误用。

§2.5 比较运算:谁可以 ==

Go 把"可比较性"写进了类型系统。能用 `==` / `!=` 比较的:数值、字符串(按字节比较,与 C 的 `strcmp` 对应但更直接)、布尔、指针、channel、数组与结构体(要求其元素/字段都可比较)。**切片、map、函数不可比较**(只能与 `nil` 比),这是编译期错误。关系运算 `<` `>` 只对数值、字符串、有序数组/结构体有效。

示例 2-6 比较运算规则

```
package main

import "fmt"

func main() {
    // 字符串按字节序比较(字典序)
    fmt.Println("apple" < "banana") // true

    // 结构体整体比较:所有字段可比较即可
    type point struct{ x, y int }
    p1 := point{1, 2}
    p2 := point{1, 2}
    fmt.Println(p1 == p2) // true

    // 数组整体比较
    fmt.Println([2]int{1, 2} == [2]int{1, 2}) // true

    // 以下全部编译失败
    // a := []int{1}
    // fmt.Println(a == a)           // 切片不可比较
    // fmt.Println(map[string]int{} == nil) // map 只能与 nil 比?
    // fmt.Println(map[string]int(nil) == nil) // 这行是合法的,但与 map 值无关

    // NaN 的坑:NaN 不等于任何值,包括它自己
    nan := 0.0 / 0.0
    fmt.Println(nan == nan) // false
    fmt.Println(nan < 1.0)  // false!任何比较都是 false

    // 判断 NaN 的正确姿势
    if nan != nan {
        fmt.Println("检测到 NaN")
    }
}
```

⚠ 易错点 2-4:NaN 的"任何比较都是 false"

`nan == nan` 为 `false`, `nan < 1` 也为 `false`。若拿 `v < min` 判断最小值,遇到 NaN 会漏掉。判断 NaN 用 `v != v` 或 `math.IsNaN(v)`;排序、求极值时先过滤 NaN。这与 C 的浮点行为一致,但 Go 程序员更容易踩中,因为 Go 鼓励直接比较浮点结果。

§2.6 赋值与多重赋值:没有三目运算符

Go 没有 `?:`。C 里 `max = a > b ? a : b`,Go 里要写完整 `if-else`。这不是遗漏,而是官方刻意决定:三目运算符经常被嵌套滥用,而 `if-else` 每行只表达一件事。生产中的替代:

示例 2-7 没有三目:用 `if-else` 表达

```

package main

import "fmt"

func max(a, b int) int {
    if a > b {
        return a
    }
    return b
}

func main() {
    // 三目运算符的经典场景:条件赋值
    score := 85
    var level string
    if score >= 60 {
        level = "及格"
    } else {
        level = "不及格"
    }
    fmt.Println(level)

    // 多重赋值:函数多返回值(第 4 章),天然替代"返回值 + 输出参数"
    q, r := 17/5, 17%5
    fmt.Println("商:", q, "余:", r)

    // 声明与赋值分离的场景
    var msg string
    if score >= 90 {
        msg = "优秀"
    } else {
        msg = "加油"
    }
    fmt.Println(msg)
}

```

★ 重点:Go 刻意没有三目运算符

遇到"一行搞定"的冲动时,写三行 if-else。Go 团队在 FAQ 中的原话大意是:三目运算符经常被用来写难读的嵌套, if-else 更直白。习惯了会发现条件赋值反而更清晰——而且可以插入日志、断言,不必改写结构。

§2.7 优先级与求值顺序

Go 的优先级层级比 C 少。从高到低(同层从左到右结合):

优先级	运算符	与 C 的差异
5(最高)	* / % << >> & &^	移位与位与同级(低于乘除,高于加减)
4	+ - ^	加减与位或、异或同级
3	== != < <= > >=	与 C 相同
2	&&	与 C 相同
1(最低)		与 C 相同

最大的坑在顶层:在 Go 中,移位和按位与的优先级高于加减、与比较同级(C 里 & 低于 ==)。所以 x & 0xFF == 0 在 C 里是 x & (0xFF == 0) ——在 Go 里是 (x & 0xFF) == 0!两种语言同一行代码,两种含义。生产铁律:涉及位运算的复合条件一律加括号。

求值顺序:C 的 f() + g() 中 f 与 g 谁先调用是未定义的;Go 明确规定:同一表达式内从左到右求值。副作用(如调用带打印的函数)顺序可预测。

示例 2-8 优先级陷阱与左到右求值

```
package main

import "fmt"

func f() int { fmt.Println("f"); return 1 }
func g() int { fmt.Println("g"); return 2 }

func main() {
    x := 0xFF
    // Go 中 & 高于 == :等价于 (x & 0xFF) == 0
    fmt.Println(x&0xFF == 0) // false

    // 与 C 的语义(若按 C 优先级)不同,务必加括号
    fmt.Println((x & 0xFF) == 0) // false,含义明确

    // 求值顺序:从左到右,先打印 f 再打印 g
    v := f() + g()
    fmt.Println("结果:", v)
}
```

§2.8 特色运算符:<-、... 与类型断言

- <- :channel 的收发运算符, ch <- v 发送、v := <-ch 接收。它是 Go 语法中最独特的一元/二元双形态运算符,第 9 章详解。
- ... :切片展开, f(s...) 把切片展开为可变参数的多个实参;数组字面量 [...]int{1,2,3} 让编译器数长度。
- . 与 [] :字段/下标访问,与 C 相同;但对空指针、越界下标,Go 会 panic 而非 UB。
- 类型断言 v.(T) :第 6 章详述,是接口动态类型的"向下转型"运算符。

示例 2-9 展开运算符与 ... 数组

```

package main

import "fmt"

func sum(nums ...int) int {
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}

func main() {
    // ... 展开:把切片铺开成多个实参
    data := []int{1, 2, 3, 4}
    fmt.Println(sum(data...)) // 10

    // 编译器数长度的数组字面量
    arr := [...]int{5, 6, 7}
    fmt.Println(len(arr)) // 3
}

```

本章速查表

主题	要点
整数除法	$7/2 == 3$; 取模 $-5\%3 == -2$ (截断向零,与 C 一致)
溢出	定义良好的回绕(补码),不是 UB;生产上仍要防回绕 bug
除零	整数除零运行期 panic;浮点除零得 Inf/NaN,不 panic
自增自减	只有 <code>i++</code> / <code>i--</code> 语句;不能作表达式; <code>++i</code> 不存在
位运算	<code>&</code> <code>^</code> (取反/异或) <code>&^</code> (位清除) <code><<</code> <code>>></code>
右移语义	无符号逻辑右移;有符号算术右移(补符号位)
逻辑运算	<code>&&</code> / <code> </code> 短路;操作数必须是 bool,无隐式真值
比较	数值/字符串/指针/数组/结构体可比较;slice、map、函数不可比较
NaN	<code>nan != nan</code> ;任何比较都 false;用 <code>math.IsNaN</code> 判断
赋值	<code>a, b = b, a</code> 多重赋值; <code>:=</code> 短声明(至少一个新变量)
三目	不存在!用 if-else;不要怀念 <code>?:</code>
优先级	乘除移位位与 <code>></code> 加减位或位异或 <code>></code> 比较 <code>></code> <code>&&</code> <code>></code> <code> </code>
求值顺序	同一表达式从左到右,确定无 UB
特色运算符	<code><-</code> 通道; <code>...</code> 展开; <code>v.(T)</code> 类型断言; <code>[...]</code> 数组

最小必会清单

- 能解释"Go 的整数溢出不是 UB"与"整数除零 panic"两条规则

- 能说出 `-5%3`、`7/2`、`1.0/0.0` 的结果
- 能解释 `i++` 为什么不能出现在表达式中
- 能写出 `&^` 清除标志位、`^` 取反、有符号右移的示例
- 能解释 `if x` 为什么在 Go 中是编译错误
- 能说出哪四种类型不可用 `==` 比较
- 能写出无三目运算符时的条件赋值写法
- 能默写 Go 运算符优先级表,并解释 `x&0xFF == 0` 在 Go 中的含义

自测题

Q 1. 计算下列表达式的值:(a) `9/2` ;(b) `-9%4` ;(c) `9%(-4)` ;(d) `0.9/0.3`

✓ 参考答案

(a) 4(整数除法截断);(b) -1(截断向零: $-9 = -2 \times 4 - 1$);(c) 1($9 = -2 \times (-4) + 1$, 余数符号取被除数);(d) 约 2.9999999999999996——浮点误差,永远不要用 `==` 比较浮点运算结果,应比较差值绝对值 `math.Abs(a-b) < 1e-9`。

Q 2. 下面哪几行能编译? `x := 5; x++`、`y := 5; y++ + 1`、`z := 5; ++z`、`w := 5; w = w + 1`

✓ 参考答案

第 1 行与第 4 行能编译。`y++ + 1` 与 `++z` 都是语法错误——自增只能是独立语句,且只有后缀形式。

Q 3. 权限掩码 `perm := 0b110`,用 `&^` 清除读权限(位 0)后, `perm` 的值是多少?

✓ 参考答案

`0b110 &^ 0b001 = 0b110`——位 0 本来就是 0,没变化。`&^` 只把"左操作数为 1 且右操作数为 1"的位清零,其余位保留。若清除写权限(位 1): `0b110 &^ 0b010 = 0b100`。

Q 4. 判断对错:(a) 切片可以用 `==` 与另一个切片比较;(b) 两个字段都相等的结构体可以用 `==` 比较;(c) `map` 可以与 `nil` 比较;(d) 函数可以赋给变量。

✓ 参考答案

(a) 错,切片不可比较(编译错误),只能与 `nil` 比较;(b) 对,若所有字段可比较;(c) 对, `m == nil` 合法;(d) 对,函数是一等公民(第 4 章),但函数值之间不能比较。

Q 5. 写出在 Go 中 "`max = a > b ? a : b`" 的等价写法,并说明 Go 为什么没有三目运算符。

✓ 参考答案

`if a > b { max = a } else { max = b }`。Go 官方认为三目运算符易被嵌套滥用, `if-else` 每行只表达一件事,可读性更好。这也让条件分支可以自然插入日志与断言。

Q 6. 表达式 `x & 0xFF == 0` 在 C 和 Go 中分别怎么解析?结果有何不同?

✅ 参考答案

C 中 `==` 优先级高于 `&`:解析为 `x & (0xFF == 0)`,即 `x & 0`,恒为 0。Go 中 `&` 优先级高于 `==`:解析为 `(x & 0xFF) == 0`,判断低 8 位是否全零。同一行代码两种含义——跨语言维护时务必加括号。

章末实验题:表达式求值计算器

📄 实验 2:表达式求值计算器

实验目标:实现一个命令行表达式计算器,支持加减乘除、取模、括号、位运算(与/或/异或/移位)与一元负号,正确处理优先级。

实验要求:

- 任务 1:用递归下降法实现优先级分层:加减 < 乘除取模 < 一元负号 < 括号(覆盖:\$2.7 优先级)
- 任务 2:支持 `&` `|` `^` `<<` `>>` 位运算,并让位运算优先级低于加减、高于比较(覆盖:\$2.3 位运算、优先级)
- 任务 3:解析表达式时验证 `-5%3` 的截断语义(覆盖:\$2.1 取模)
- 任务 4:处理除零:用 `panic/recover` 捕获并给出友好错误(覆盖:\$2.1 除零 `panic`)
- 任务 5:支持十六进制/二进制字面量 `0xFF`、`0b1010` (覆盖:\$2.3 字面量位运算)
- 任务 6:程序主体用多重赋值与短声明,不用任何 `++` 表达式(覆盖:\$2.2、\$2.6)

实验环境:Go 1.21+, `go run main.go`,输入 `go vet main.go` 确认无警告。

第一步 把表达式切分成记号(token):用 `strings.Replacer` 给每个运算符加空格,再 `strings.Fields` 切分;注意 1 << 2 要切出 1、<<、2

第二步 现解析器结构体: `toks []string` 与 `pos int`,提供 `peek()` / `next()` 两个方法

第三步 最低优先级开始写 `parseOr`,层层向下调用,每层只处理自己的运算符并循环

第四步 一元负号用"遇到 `-` 就递归调用 `parseUnary` 并取反";括号直接递归到最高层

第五步 `strconv.ParseInt(t, 0, 64)` 解析数字,自动支持 `0x`、`0b` 前缀

第六步 `eval` 里 `defer + recover` 捕获除零 `panic`,返回错误消息(第 9 章会系统讲 `panic/recover`)

✔ 参考答案要点

```
// 表达式求值计算器:递归下降 + 优先级分层
package main

import (
    "fmt"
    "strconv"
    "strings"
)

type parser struct {
    toks []string
    pos  int
}

func (p *parser) next() string {
    if p.pos < len(p.toks) {
        t := p.toks[p.pos]
        p.pos++
        return t
    }
    return ""
}

func (p *parser) peek() string {
    if p.pos < len(p.toks) {
        return p.toks[p.pos]
    }
    return ""
}

// 最低优先级:按位或
func (p *parser) parseOr() int {
    v := p.parseXor()
    for p.peek() == "|" {
        p.next()
        v = v | p.parseXor()
    }
    return v
}

// 异或
func (p *parser) parseXor() int {
    v := p.parseAnd()
    for p.peek() == "^" {
        p.next()
        v = v ^ p.parseAnd()
    }
    return v
}

// 按位与
func (p *parser) parseAnd() int {
    v := p.parseShift()
    for p.peek() == "&" {
        p.next()
    }
}
```

```

        v = v & p.parseShift()
    }
    return v
}

// 移位
func (p *parser) parseShift() int {
    v := p.parseAdd()
    for {
        op := p.peek()
        if op == "<<" || op == ">>" {
            p.next()
            r := p.parseAdd()
            if op == "<<" {
                v = v << r
            } else {
                v = v >> r
            }
        }
        continue
    }
    break
}
return v
}

// 加减
func (p *parser) parseAdd() int {
    v := p.parseMul()
    for {
        op := p.peek()
        if op == "+" || op == "-" {
            p.next()
            r := p.parseMul()
            if op == "+" {
                v += r
            } else {
                v -= r
            }
        }
        continue
    }
    break
}
return v
}

// 乘除取模
func (p *parser) parseMul() int {
    v := p.parseUnary()
    for {
        op := p.peek()
        if op == "*" || op == "/" || op == "%" {
            p.next()
            r := p.parseUnary()
            switch op {
                case "*":

```

```

        v *= r
    case "/":
        v /= r // 除零 panic,由 eval 的 recover 捕获
    case "%":
        v %= r // 截断语义:-5 % 3 == -2
    }
    continue
}
break
}
return v
}

// 一元负号与括号
func (p *parser) parseUnary() int {
    if p.peek() == "-" {
        p.next()
        return -p.parseUnary()
    }
    if p.peek() == "(" {
        p.next()
        v := p.parseOr()
        p.next() // 吃掉右括号
        return v
    }
    t := p.next()
    n, err := strconv.ParseInt(t, 0, 64) // 基数 0:自动识别 0x/0b/0o
    if err != nil {
        panic("无法解析:" + t)
    }
    return int(n)
}

// eval 计算表达式;错误以 panic 形式抛出,在此恢复
func eval(expr string) (result int, errMsg string) {
    defer func() {
        if r := recover(); r != nil {
            errMsg = fmt.Sprintf(r)
        }
    }()
    rep := strings.NewReplacer("(", " ( ", ")", " ) ", "*", " * ",
        "/", " / ", "%", " % ", "+", " + ", "-", " - ",
        "&", " & ", "|", " | ", "^", " ^ ", "<<", " << ", ">>", " >> ")
    p := &parser{toks: strings.Fields(rep.Replace(expr))}
    result = p.parseOr()
    return
}

func main() {
    exprs := []string{
        "1 + 2 * 3",
        "(1 + 2) * 3",
        "10 / 3",
        "7 % 3",
        "-5 % 3",
    }
}

```

```

    "1 << 4",
    "0xF & 0b1010",
    "1 + 2 << 2",
    "1 / 0",
}
for _, e := range exprs {
    v, err := eval(e)
    if err != "" {
        fmt.Printf("%-14s => 错误: %s\n", e, err)
    } else {
        fmt.Printf("%-14s => %d\n", e, v)
    }
}
}

```

设计说明:每个优先级一个函数,函数之间只通过"循环 + 调用下一层"连接,结构即文法,天然支持任意嵌套括号;位运算层插在加减与比较之间,复现 Go 的优先级规则; `eval` 用命名返回值 + `defer/recover` 把 `panic` 转成错误字符串,演示了"Go 没有异常但异常性错误可控"的理念(第 9 章展开)。实验时请先实现到加减乘除,再逐层加入位运算,每加一层都用速查表的优先级验证一遍。

下一章预告:第3章 流程控制。表达式的规矩学完了,下一章看流程:`switch` 没有 `fallthrough`、`for` 是唯一的循环、`range` 的遍历语义、带标签的 `break/continue` 与 `goto` 的合法用法——每一条都会颠覆 C 程序员的手感。